SEGUNDO DEPARTAMENTAL- EJERCICIO 18- ORDENAMIENTO (PE)

En este programa desarrollamos un sistema de ordenamiento utilizando el algoritmo Quick Sort, el cual permite organizar diferentes tipos de datos de manera ascendente. Para ello, implementamos tres funciones sobrecargadas llamadas quickSort(), cada una encargada de ordenar un tipo de información diferente: números enteros, caracteres y registros de personas según su edad. También creamos una estructura llamada Persona, que permite almacenar el nombre y la edad de cada individuo, además, utilizamos arreglos para guardar los elementos, ciclos para capturar y mostrar la información, y llamadas recursivas para realizar el proceso de ordenamiento, de esta manera, aplicamos conceptos como estructuras, funciones, sobrecarga de funciones, recursividad y manejo de arreglos.

Código Completo:

```
#include <iostream>
#include <string>
using namespace std;

// Nuevo tipo de dato
struct Persona {
    string nombre;
    int edad;
};

// Quick Sort para enteros
void quickSort(int v[], int inicio, int fin) {
    int i = inicio, j = fin;
    int pivote = v[(inicio + fin) / 2];

    while (i <= j) {
        while (v[i] < pivote) i++;
        while (v[j] > pivote) j--;

        if (i <= j) {
            int aux = v[i];
            v[i] = v[j];
            v[j] = aux;
            i++;
            j--;
        }
    }

    if (inicio < j) quickSort(v, inicio, j);
    if (i < fin) quickSort(v, i, fin);
}

// Quick Sort para caracteres
void quickSort(char v[], int inicio, int fin) {
    int i = inicio, j = fin;
    char pivote = v[(inicio + fin) / 2];

    while (i <= j) {
        while (v[i] < pivote) i++;
        while (v[j] > pivote) j--;

        if (i <= j) {
            char aux = v[i];
```

```
            v[i] = v[j];
            v[j] = aux;
            i++;
            j--;
        }
    }

    if (inicio < j) quickSort(v, inicio, j);
    if (i < fin) quickSort(v, i, fin);
}

// Quick Sort para el nuevo tipo Persona
void quickSort(Persona v[], int inicio, int fin) {
    int i = inicio, j = fin;
    int pivote = v[(inicio + fin) / 2].edad;

    while (i <= j) {
        while (v[i].edad < pivote) i++;
        while (v[j].edad > pivote) j--;

        if (i <= j) {
            Persona aux = v[i];
            v[i] = v[j];
            v[j] = aux;
            i++;
            j--;
        }
    }

    if (inicio < j) quickSort(v, inicio, j);
    if (i < fin) quickSort(v, i, fin);
}

int main() {
    int opcion, n;

    cout << "=== QUICK SORT EN PE ===" << endl;
    cout << "1. Ordenar enteros" << endl;
    cout << "2. Ordenar caracteres" << endl;
    cout << "3. Ordenar personas por edad" << endl;
    cout << "Elige una opcion: ";
    cin >> opcion;

    cout << "Cantidad de elementos (1 a 100): ";
    cin >> n;

    if (n < 1 || n > 100) {
        cout << "Cantidad no valida." << endl;
        return 0;
    }

    if (opcion == 1) {
        int v[100];

        for (int i = 0; i < n; i++) {
            cout << "Entero " << i + 1 << ": ";
            cin >> v[i];
        }

        quickSort(v, 0, n - 1);

        cout << "Enteros ordenados: ";
        for (int i = 0; i < n; i++)
```

```
            cout << v[i] << " ";
    }
    else if (opcion == 2) {
        char v[100];

        for (int i = 0; i < n; i++) {
            cout << "Caracter " << i + 1 << ": ";
            cin >> v[i];
        }

        quickSort(v, 0, n - 1);

        cout << "Caracteres ordenados: ";
        for (int i = 0; i < n; i++)
            cout << v[i] << " ";
    }
    else if (opcion == 3) {
        Persona v[100];

        for (int i = 0; i < n; i++) {
            cout << "Nombre de la persona " << i + 1 << ": ";
            cin >> v[i].nombre;
            cout << "Edad: ";
            cin >> v[i].edad;
        }

        quickSort(v, 0, n - 1);

        cout << "\nPersonas ordenadas por edad:\n";
        for (int i = 0; i < n; i++)
            cout << v[i].nombre << " - "
                 << v[i].edad << " anios" << endl;
    }
    else {
        cout << "Opcion no valida." << endl;
    }

    return 0;
}
```

Explicación del Código:

1. Definición de la Estructura Persona (struct Persona)

Creamos una estructura llamada Persona, que funciona como un nuevo tipo de dato definido por el usuario. Su objetivo es agrupar la información de cada individuo dentro de un mismo registro, permitiendo almacenar tanto su nombre como su edad.

2. Función Quick Sort para Números Enteros (void quickSort(int v[], int inicio, int fin))

Es la primera función de ordenamiento del programa y se encarga de organizar un arreglo de números enteros de menor a mayor. Recibe tres parámetros: el arreglo v[], la posición inicial inicio y la posición final fin. Dentro de la función utilizamos dos índices, i y j, para recorrer el arreglo, además de seleccionar un elemento central como pivote. Posteriormente, comparamos los números con el pivote e intercambiamos sus

posiciones cuando es necesario. Finalmente, utilizamos llamadas recursivas para ordenar las partes restantes del arreglo.

3. Función Quick Sort para Caracteres (void quickSort(char v[], int inicio, int fin))

Es la segunda función de ordenamiento y realiza el mismo procedimiento que la anterior, pero trabaja con caracteres en lugar de números enteros. Recibe un arreglo de tipo char, selecciona un carácter central como pivote y compara los demás caracteres para determinar su posición, también utiliza los índices i y j, una variable auxiliar para realizar intercambios y llamadas recursivas para completar el ordenamiento, de esta manera, podemos organizar los caracteres de acuerdo con su valor dentro del conjunto de caracteres utilizado por el sistema.

4. Función Quick Sort para el Tipo Persona (void quickSort(Persona v[], int inicio, int fin))

Es la tercera función de ordenamiento y permite organizar un arreglo de personas tomando como referencia su edad. A diferencia de las funciones anteriores, trabaja con elementos de tipo Persona, por lo que las comparaciones se realizan utilizando el atributo edad de cada registro, cuando es necesario intercambiar elementos, se intercambian las estructuras completas, conservando la relación entre el nombre y la edad de cada persona.

5. Función Principal (int main())

Es el punto de entrada del programa. Dentro de ella declaramos las variables opcion y n, que permiten controlar el tipo de ordenamiento y la cantidad de elementos que se van a procesar, también contiene el menú de opciones, la validación de la cantidad de elementos y las instrucciones necesarias para capturar, ordenar y mostrar la información.

6. Sobrecarga de Funciones (quickSort())

Una de las características principales del programa es la sobrecarga de funciones. Esto significa que podemos utilizar el mismo nombre para varias funciones, siempre que sus parámetros sean diferentes, en este caso, tenemos tres funciones llamadas quickSort(): una trabaja con números enteros, otra con caracteres y otra con estructuras de tipo Persona, el compilador identifica cuál debe ejecutarse dependiendo del tipo de arreglo que recibe.

SEGUNDO DEPARTAMENTAL- EJERCICIO 18- ORDENAMIENTO (POO)

En este programa desarrollamos un sistema de ordenamiento utilizando el paradigma de Programación Orientada a Objetos (POO) y el algoritmo Quick Sort. El objetivo es organizar diferentes tipos de datos de manera ascendente, como números enteros, caracteres y objetos que contienen información de personas, tomando como referencia su edad. Para ello, creamos dos clases: Persona y Ordenador. La primera se encarga de almacenar y mostrar los datos de cada individuo, mientras que la segunda contiene las funciones necesarias para realizar el ordenamiento de los diferentes tipos de datos, además, utilizamos constructores para inicializar los objetos, arreglos para almacenar la información, ciclos para capturar y mostrar los elementos y funciones recursivas para ejecutar el algoritmo.

Código Completo:

```
#include <iostream>
#include <string>
using namespace std;

// Clase que representa un nuevo tipo de dato
class Persona {
public:
    string nombre;
    int edad;

    Persona() {
        nombre = "";
        edad = 0;
    }

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }

    void mostrar() {
        cout << nombre << " - " << edad << " anios" << endl;
    }
};

// Clase encargada del ordenamiento
class Ordenador {
public:

    // Quick Sort para enteros
    void quickSort(int v[], int inicio, int fin) {
        int i = inicio, j = fin;
        int pivote = v[(inicio + fin) / 2];

        while (i <= j) {
            while (v[i] < pivote) i++;
            while (v[j] > pivote) j--;

            if (i <= j) {
                int aux = v[i];
                v[i] = v[j];
                v[j] = aux;
```

```
                i++;
                j--;
            }
        }

        if (inicio < j) quickSort(v, inicio, j);
        if (i < fin) quickSort(v, i, fin);
    }

    // Quick Sort para caracteres
    void quickSort(char v[], int inicio, int fin) {
        int i = inicio, j = fin;
        char pivote = v[(inicio + fin) / 2];

        while (i <= j) {
            while (v[i] < pivote) i++;
            while (v[j] > pivote) j--;

            if (i <= j) {
                char aux = v[i];
                v[i] = v[j];
                v[j] = aux;
                i++;
                j--;
            }
        }

        if (inicio < j) quickSort(v, inicio, j);
        if (i < fin) quickSort(v, i, fin);
    }

    // Quick Sort para objetos Persona
    void quickSort(Persona v[], int inicio, int fin) {
        int i = inicio, j = fin;
        int pivote = v[(inicio + fin) / 2].edad;

        while (i <= j) {
            while (v[i].edad < pivote) i++;
            while (v[j].edad > pivote) j--;

            if (i <= j) {
                Persona aux = v[i];
                v[i] = v[j];
                v[j] = aux;
                i++;
                j--;
            }
        }

        if (inicio < j) quickSort(v, inicio, j);
        if (i < fin) quickSort(v, i, fin);
    }
};

int main() {
    Ordenador ordenador;
    int opcion, n;

    cout << "=== QUICK SORT EN POO ===" << endl;
    cout << "1. Ordenar enteros" << endl;
    cout << "2. Ordenar caracteres" << endl;
    cout << "3. Ordenar personas por edad" << endl;
    cout << "Elige una opcion: ";
```

```
    cin >> opcion;

    cout << "Cantidad de elementos (1 a 100): ";
    cin >> n;

    if (n < 1 || n > 100) {
        cout << "Cantidad no valida." << endl;
        return 0;
    }

    if (opcion == 1) {
        int v[100];

        for (int i = 0; i < n; i++) {
            cout << "Ingresa el entero " << i + 1 << ": ";
            cin >> v[i];
        }

        ordenador.quickSort(v, 0, n - 1);

        cout << "Enteros ordenados: ";
        for (int i = 0; i < n; i++)
            cout << v[i] << " ";
    }
    else if (opcion == 2) {
        char v[100];

        for (int i = 0; i < n; i++) {
            cout << "Ingresa el caracter " << i + 1 << ": ";
            cin >> v[i];
        }

        ordenador.quickSort(v, 0, n - 1);

        cout << "Caracteres ordenados: ";
        for (int i = 0; i < n; i++)
            cout << v[i] << " ";
    }
    else if (opcion == 3) {
        Persona v[100];

        for (int i = 0; i < n; i++) {
            string nombre;
            int edad;

            cout << "Nombre: ";
            cin >> nombre;
            cout << "Edad: ";
            cin >> edad;

            v[i] = Persona(nombre, edad);
        }

        ordenador.quickSort(v, 0, n - 1);

        cout << "\nPersonas ordenadas por edad:\n";
        for (int i = 0; i < n; i++)
            v[i].mostrar();
    }
    else {
        cout << "Opcion no valida." << endl;
    }
```

```
    return 0;
}
```

Explicación del Código:

1. Definición de la Clase Persona (class Persona)

Creamos una clase llamada Persona, que funciona como una plantilla para representar a los individuos que se registran en el programa. Su objetivo es almacenar la información de cada persona y proporcionar un método para mostrar sus datos, esta clase permite trabajar con objetos que contienen información relacionada, en lugar de manejar cada dato de manera independiente.

2. Constructor por Defecto (Persona())

Creamos un constructor sin parámetros que se encarga de inicializar los atributos de la clase cuando se crea un objeto sin proporcionar información, este constructor permite crear objetos Persona con valores iniciales definidos.

3. Constructor con Parámetros (Persona(string n, int e))

Creamos un segundo constructor que recibe dos parámetros: n, correspondiente al nombre, y e, correspondiente a la edad. Dentro del constructor asignamos estos valores a los atributos nombre y edad. Su función es permitir la creación de objetos Persona proporcionando directamente la información de cada individuo, sin necesidad de asignar los atributos por separado.

4. Método para Mostrar los Datos de Persona (void mostrar())

Es el método encargado de presentar en pantalla la información almacenada en cada objeto de tipo Persona, mediante cout mostramos el nombre y la edad del individuo. Este método permite consultar los datos de cada persona de manera organizada y evita tener que escribir las mismas instrucciones de salida en diferentes partes del programa.

5. Definición de la Clase Ordenador (class Ordenador)

Creamos una clase llamada Ordenador, cuya función principal es realizar el ordenamiento de los diferentes tipos de datos mediante el algoritmo Quick Sort. Dentro de esta clase declaramos tres métodos con el mismo nombre quickSort(), pero con diferentes parámetros, de esta manera, podemos utilizar un mismo algoritmo para ordenar números enteros, caracteres y objetos de tipo Persona.

6. Función Quick Sort para Números Enteros (void quickSort(int v[], int inicio, int fin))

Es el primer método de ordenamiento de la clase Ordenador y se encarga de organizar un arreglo de números enteros de menor a mayor, recibe tres parámetros: el arreglo v[], la posición inicial inicio y la posición final fin. Dentro del método utilizamos dos índices, i

y j, para recorrer el arreglo, además de seleccionar un elemento central como pivote, después, comparamos los números con el pivote e intercambiamos sus posiciones cuando es necesario y finalmente, utilizamos llamadas recursivas para ordenar las partes restantes del arreglo.

7. Función Quick Sort para Caracteres (void quickSort(char v[], int inicio, int fin))

Es el segundo método de ordenamiento y realiza el mismo procedimiento que el anterior, pero trabaja con caracteres en lugar de números enteros. Recibe un arreglo de tipo char, selecciona un carácter central como pivote y compara los demás caracteres para determinar su posición, también utiliza los índices i y j, una variable auxiliar llamada aux para realizar los intercambios y llamadas recursivas para completar el ordenamiento. De esta manera, podemos organizar los caracteres de acuerdo con su valor dentro del conjunto de caracteres utilizado por el sistema.

8. Función Quick Sort para Objetos Persona (void quickSort(Persona v[], int inicio, int fin))

Es el tercer método de ordenamiento y permite organizar un arreglo de objetos de tipo Persona, tomando como referencia la edad de cada individuo, a diferencia de los métodos anteriores, las comparaciones se realizan utilizando el atributo edad de cada objeto, cuando es necesario intercambiar elementos, se intercambian los objetos completos, conservando la relación entre el nombre y la edad de cada persona.

9. Ordenamiento de Números Enteros (if (opcion == 1))

Cuando el usuario selecciona la primera opción, declaramos un arreglo llamado v de tipo int con capacidad para cien números, después, utilizamos un ciclo for para solicitar cada número y almacenarlo en el arreglo. Una vez capturados todos los datos, llamamos al método ordenador.quickSort(v, 0, n - 1) para ordenarlos de menor a mayor. Finalmente, utilizamos otro ciclo for para mostrar los números ordenados.

10. Ordenamiento de Caracteres (else if (opcion == 2))

Cuando el usuario selecciona la segunda opción, declaramos un arreglo de tipo char con capacidad para cien caracteres, mediante un ciclo for solicitamos los caracteres y los almacenamos en sus respectivas posiciones, después, llamamos al método ordenador.quickSort(v, 0, n - 1) correspondiente al tipo de dato char para realizar el ordenamiento y finalmente, mostramos los caracteres organizados de manera ascendente.

11. Ordenamiento de Personas por Edad (else if (opcion == 3))

Cuando el usuario selecciona la tercera opción, declaramos un arreglo llamado v de tipo Persona, con capacidad para cien objetos, dentro del ciclo for declaramos dos variables: nombre, de tipo string, y edad, de tipo int, después, solicitamos los datos de cada individuo y utilizamos la instrucción v[i] = Persona(nombre, edad) para crear un objeto con la información capturada y almacenarlo en la posición correspondiente del

arreglo. Una vez registrados todos los individuos, llamamos al método ordenador.quickSort(v, 0, n - 1) para organizar las personas de menor a mayor edad y finalmente, utilizamos otro ciclo for y el método v[i].mostrar() para presentar la información de cada persona.

12. Sobrecarga de Métodos (quickSort())

Una de las características principales del programa es la sobrecarga de métodos. Esto significa que podemos utilizar el mismo nombre para varios métodos dentro de una misma clase, siempre que sus parámetros sean diferentes, en este caso, la clase Ordenador contiene tres métodos llamados quickSort(): uno trabaja con números enteros, otro con caracteres y otro con objetos de tipo Persona. El compilador identifica cuál debe ejecutarse dependiendo del tipo de arreglo que recibe el objeto ordenador.